

Disclaimer: Dieses OER-Dokument stammt aus dem Projekt [proTirone](#), das über GitHub [freie Unterrichtsmaterialien](#) samt [Quellcode](#) offeriert. proTirone-Materialien werden unter den Bedingungen der [CC-BY-4.0-Lizenz](#) so angeboten, wie sie sind, ohne Zusage bestimmter Eigenschaften und ohne Gewährleistung (§5). Dafür dürfen sie – bei angemessener Namensnennung (§3) – für beliebige (auch kommerzielle) Zwecke verändert und weitergegeben werden (§2).

LF11c:90:Ersatzleistung:Converter

Aufgabe **LF11c:Klausurersatzleistung:Allgemein**

Sie sollen in Python einen Konverter von einem Input-Format ins JSON-Format programmieren. Die folgenden Teilaufgaben brechen dieses Ziel in Teilschritte herunter. Was davon bewertet wird, ist durch die erreichbaren Punkte ausgewiesen. Alle anderen Unterpunkte - insbesondere die Befolgung des *Minimal-Viable-Product-Prinzips* - sollen Ihnen die Arbeit nur erleichtern.

Laden Sie zum Schluss Ihr lauffähiges Programm in nur einer Programmdatei (Endung `.py`) als Aufgabenlösung hoch.

Falls Sie darüberhinaus

- etwas beschreiben sollen, fügen Sie Ihren Text als Kommentar in Ihren Programmcode ein.
- die einzulesende Daten vor der Verarbeitung modifizieren müssen, laden Sie zum Schluss auch Ihre Version der Input-Datendatei hoch. (Extension typgemäß).
- Testdaten zusammenstellen sollen, laden Sie zum Schluss die als Test-Input-Datendatei hoch (Extension typgemäß).

Generieren Sie am Ende im üblichen Übungsbordner Ihren eigenen Unterordner `YYYY-MM-DD-conv-teta-IHRNAME`. Laden Sie all Ihre Dateien dahinein hoch

Es sind 32 Punkte erreichbar (plus 4 Zusatzpunkte)

Punkte	Zensur
32	1
31	1,2
30	1,4
29	1,6
28	1,9

Punkte	Zensur
27	2,1
26	2,4
...	...

Fortsetzung gemäß [ihk-notenschluessel](#)

Aufgabe LF11c:Klausurersatzleistung:Step:01

- ☐ Laden Sie die Datei [dsp.teta.xyz](#) herunter.
- ☐ Machen Sie sich mit den Daten vertraut.
- ☐ Finden Sie die Ordnung der Daten.
- ☐ Modifizieren Sie die Daten so, dass jeder Datensatz in einer eigenen Zeile steht. **(2P)**
- ☐ Beschreiben Sie die Datensatzstruktur. **(1P)**

Lösung zu Step 01: [→ ZP:Sheet:2]

Aufgabe LF11c:Klausurersatzleistung:Step:02

- ☐ Bereinigen Sie die Daten syntaktisch, wo es die Einheitlichkeit gebietet. **(2P)**
- ☐ Beschreiben Sie, welche Datensätze wie und warum syntaktisch bereinigt werden mussten. **(1P)**

Lösung zu Step 02: [→ ZP:Sheet:2]

Aufgabe LF11c:Klausurersatzleistung:Step:03

- ☐ Bereinigen Sie die Daten semantisch, wo es die Konsistenz gebietet. **(2P)**
- ☐ Beschreiben Sie, welche Datensätze wie und warum semantisch bereinigt werden mussten. **(1P)**

Lösung zu Step 03: [→ ZP:Sheet:3]

Aufgabe LF11c:Klausurersatzleistung:Step:04

- ☐ Programmieren Sie jetzt in Python einen Konverter, der die (syntaktisch und semantisch bereinigten) Inputdaten in dem Format aus einer Datei einliest, das Sie selbst in Step 1 festgelegt haben (= im Folgenden: *Step1Format*). Als Erstes soll er sie auch nur in diesem *Step1Format* eine andere Datei zurückschreiben:
 - ☐ **MVP:**
 - ☐ Extrahieren Sie 3 Datensätze aus den von Ihnen bereinigten Inputdaten als Testdaten. **(1P)**
 - ☐ Definieren Sie eine Zwischenrepräsentation.
 - ☐ Initialisieren Sie - diesem Format entsprechend - in Ihrem Programm *hard-gecodetet Testdaten* mit 2 Datensätzen aus Ihren Testdaten. **(2P)**
 - ☐ Schreiben Sie das Programmskelett mit 'leeren Funktionen'
 - ☐ **VP-0.1:**
 - ☐ Programmieren Sie einen Output-Adapter für Ihren Konverter, der die *hard-gecodeten Testdaten* im *Step1Format* in eine Datei schreibt. **(4P)**
 - ☐ Programmieren Sie einen Input-Adapter des Konverters, der Ihre Testdaten einliest und in eine interne Datenstruktur gemäß Ihrer Zwischenrepräsentation umwandelt. **(4P)**
 - ☐ Überprüfen Sie dann, ob Ihr Programm auch die gesamten bereinigten Inputdaten im *Step1Format* einlesen und im *Step1Format* in eine andere Datei wieder ausgibt.
 - ☐ Kommentieren Sie Ihr Programm kurz, aber aussagekräftig. **(1P)**

Lösung zu Step 04: [→ ZP:Sheet:4] & [→ ZP:Sheet:5]

Aufgabe LF11c:Klausurersatzleistung:Step:05

- ☐ Programmieren Sie in Python einen Konverter, der bereinigten Inputdaten [*Step1Format*] aus einer Datei einliest und in anderen Formate in eine andere Datei ausgibt.
 - ☐ **VP-0.2:**
 - ☐ Programmieren Sie einen zweiten Output-Adapter für Ihren Konverter, der die *hard-gecodetet Testdaten* im CSV-Format in eine Datei schreibt. **(1P)**
 - ☐ **VP-1.0:**
 - ☐ Programmieren Sie einen dritten Output-Adapter für Ihren Konverter, der die *hard-gecodetet Testdaten* im JSON-Format in eine Datei schreibt. **(4P)**

- ☐ Modifizieren Sie Ihren Konverter so, dass er seinen Nutzer das Ausgabeformat wählen lässt. **(1P)**
- ☐ Überprüfen Sie dann, ob Ihr Programm auch die gesamten bereinigte Inputdaten [Step1Format] einlesen und im JSON-Format und im CSV-Format in eine andere Datei ausgeben kann.
- ☐ Kommentieren Sie Ihr Programm kurz, aber aussagekräftig. **(1P)**

Lösung zu Step 05: [→ ZP:Sheet:6] & [→ ZP:Sheet:7] & [→ ZP:Sheet:8]

Aufgabe LF11c:Klausurersatzleistung:Step:06

- ☐ Ergänzen Sie Ihren Konverter um einen Output-Adapter, der die Datenmenge (Anzahl der Bytes) prinzipiell ohne Datenverlust deutlich reduziert.
 - **VP-1.1:**
 - ☐ Beschreiben Sie Ihre Verbesserungsideen. **(2P)**
 - ☐ Programmieren Sie dafür exemplarisch einen vierten Output-Adapter. **(2P)**

Lösung zu Step 06: [→ ZP:Sheet:9]

1. Enkodiere den Shall- und den is-Wert in dasselbe 2-Byte-Integer
 2. Schreibe die Shall-is-Integer in die kondensierte csv-Datei
-

Aufgabe LF11c:Klausurersatzleistung:Step:07

- ☐ Beschreiben Sie, wovon die Daten in `dsp.teta.xyz` handeln, will sagen: welchen Weltausschnitt sie erfassen. (2ZP)
- ☐ Beschreiben Sie eine auffällige Eigenart der beteiligten Person. (2ZP)

Lösung zu Step 07:

- [→ ZP:Sheet:10]
- Auffälligkeit: Nutzer startet hoch, überfordert sich, kann Level nicht halten.